

ONEDRAFT

CAD-AI — Aria Powered

Configuration, Secrets & Environment Management

Version 1.0

Status: Implementation Specification

System: OneDraft

Configuration Layer: Centralized, Environment-Aware

Secrets Layer: Secure Secret Management

Runtime Model: Immutable Configuration at Process Start

Deployment Model: Environment-Separated

Primary Environments: Development / Test / Staging / Production

1. CONFIGURATION DESIGN PRINCIPLE

OneDraft shall treat configuration as a controlled system resource rather than as arbitrary application state.

Configuration determines how the OneDraft runtime behaves within a particular environment.

The configuration system shall provide:

Environment separation

Configuration hierarchy

Secret isolation

Runtime configuration

Deployment configuration

Feature configuration

Service configuration

External provider configuration

Security configuration

Operational configuration

Configuration must be deterministic, auditable and reproducible.

2. CONFIGURATION AND SECRETS SEPARATION

Configuration and secrets are separate architectural concepts.

Configuration may contain:

- environment
- service endpoints
- feature flags
- queue names
- database hostnames
- storage locations
- logging levels
- timeouts
- worker limits
- API limits
- model configuration
- drawing defaults

Secrets may contain:

- database passwords
- API keys
- signing keys
- JWT secrets
- OAuth credentials
- encryption keys
- provider credentials
- webhook secrets
- service credentials

Secrets must never be committed to the source repository.

3. ENVIRONMENT MODEL

OneDraft shall initially define:

- development
- test
- staging
- production

Each environment shall have its own configuration boundary.

Conceptually:

OneDraft
↓
Environment
↓
Configuration

↓
Services
↓
Workers
↓
External Dependencies

Production configuration must never be implicitly inherited from development configuration.

4. DEVELOPMENT ENVIRONMENT

Development configuration shall support local engineering.

Typical services include:

PostgreSQL
Redis
Object Storage
API
Worker
Web Application
Geometry Engine
Validation Engine
Document Engine
Event Bus

Development services may run through:

Docker Compose

or an equivalent local orchestration mechanism.

Development credentials must never provide access to production resources.

5. TEST ENVIRONMENT

The test environment shall provide deterministic execution for automated testing.

It shall support:

unit tests
integration tests
API contract tests
database tests
geometry tests
validation tests
document tests
event tests
end-to-end tests

Test configuration shall use isolated resources.

Test execution must not modify production data.

6. STAGING ENVIRONMENT

Staging shall reproduce production architecture as closely as practical.

Staging shall contain:

- API
- frontend
- workers
- PostgreSQL
- queue infrastructure
- object storage
- event infrastructure
- geometry services
- validation services
- document services
- observability

Staging may use non-production credentials and data.

Production credentials must never be copied into staging.

7. PRODUCTION ENVIRONMENT

Production shall use independently managed configuration and secrets.

Production configuration shall control:

- service discovery
- database connectivity
- worker concurrency
- queue configuration
- storage
- AI providers
- authentication
- authorization
- rate limits
- observability
- backup systems
- external integrations

Production configuration changes shall be controlled and auditable.

8. CONFIGURATION HIERARCHY

Configuration shall resolve through an explicit precedence hierarchy.

The initial hierarchy shall be:

1. Built-in application defaults
2. Environment configuration
3. Deployment configuration
4. Service configuration
5. Runtime-injected configuration
6. Secret provider values

Higher-priority configuration must override lower-priority defaults only through defined mechanisms.

9. CONFIGURATION SCHEMA

Configuration shall be represented through typed schemas.

Conceptually:

```
class Settings:
    environment: str
    database: DatabaseSettings
    storage: StorageSettings
    queue: QueueSettings
    api: APISettings
    workers: WorkerSettings
    ai: AISettings
    security: SecuritySettings
    observability: ObservabilitySettings
```

Configuration validation occurs before the service becomes ready.

10. FAIL-FAST CONFIGURATION VALIDATION

A service with invalid mandatory configuration shall not enter a ready state.

Example:

```
APPLICATION START
  ↓
LOAD CONFIGURATION
  ↓
VALIDATE CONFIGURATION
  ↓
VALID
  ↓
```

START SERVICE

INVALID

↓

FAIL STARTUP

This prevents partially configured services from entering the runtime.

11. ENVIRONMENT VARIABLES

Environment variables shall provide the primary deployment-time configuration interface.

Example:

```
ONEDRAFT_ENVIRONMENT=production
ONEDRAFT_LOG_LEVEL=INFO
ONEDRAFT_API_HOST=0.0.0.0
ONEDRAFT_API_PORT=8000
ONEDRAFT_DATABASE_HOST=postgres
ONEDRAFT_DATABASE_PORT=5432
ONEDRAFT_QUEUE_URL=redis://redis:6379
```

Secret values shall not be embedded directly into source code.

12. ENVIRONMENT VARIABLE NAMING

Environment variables shall use a consistent namespace.

Primary prefix:

ONEDRAFT_

Service-specific values may use:

```
ONEDRAFT_API_
ONEDRAFT_DATABASE_
ONEDRAFT_STORAGE_
ONEDRAFT_QUEUE_
ONEDRAFT_AI_
ONEDRAFT_SECURITY_
ONEDRAFT_WORKER_
ONEDRAFT_DOCUMENT_
```

13. SECRET MANAGEMENT PRINCIPLE

Secrets shall be managed through a dedicated secret-management mechanism.

The application should retrieve secrets at deployment or startup rather than storing them in source control.

The architecture shall permit integration with:

- Docker Secrets
- Kubernetes Secrets
- cloud secret managers
- Vault-compatible systems
- environment-injected secret providers

The specific production provider remains deployment-dependent.

14. SECRET STORAGE RULE

The following are prohibited:

- hard-coded passwords
- hard-coded API keys
- hard-coded private keys
- credentials committed to Git
- credentials embedded in Docker images
- credentials embedded in frontend bundles
- credentials stored in public configuration files

Secret material must remain outside distributable application artifacts.

15. SECRET ACCESS BOUNDARY

Only services requiring a secret shall receive access to that secret.

Conceptually:

```
SECRET STORE
  ↓
SERVICE IDENTITY
  ↓
AUTHORIZED SECRET
  ↓
SERVICE
```

A frontend application shall never receive backend-only secrets.

16. SECRET EXPOSURE CONTROL

Secrets must not appear in:

- application logs
- error responses
- API responses
- telemetry
- metrics
- traces
- exception messages
- database records
- generated documents

Diagnostic systems shall redact sensitive configuration automatically.

17. SECRET ROTATION

The architecture shall support secret rotation without requiring redesign of the application.

Rotatable credentials include:

- database credentials
- AI provider credentials
- API credentials
- signing keys
- webhook secrets
- storage credentials
- integration credentials

Rotation procedures shall be documented for each production secret class.

18. KEY MANAGEMENT

Cryptographic keys shall be treated separately from ordinary configuration.

Key material may include:

- JWT signing keys
- encryption keys
- artifact signing keys
- webhook verification keys
- service identity keys

Private keys shall never be exposed to client applications.

19. DATABASE CONFIGURATION

Database configuration shall include:

- host

port
database
username
password
SSL mode
connection pool size
connection timeout
statement timeout
migration configuration

The application shall obtain these values through the configuration layer.

20. DATABASE MIGRATION CONFIGURATION

Database migrations shall be executed through the controlled migration system.

Conceptually:

APPLICATION VERSION
↓
MIGRATION VERSION
↓
DATABASE SCHEMA

A production deployment must verify migration compatibility before application rollout.

21. OBJECT STORAGE CONFIGURATION

Object storage configuration shall include:

endpoint
bucket
region
credentials
encryption mode
retention configuration
artifact namespace

The application shall never assume a specific object-storage provider.

22. QUEUE CONFIGURATION

Queue configuration shall include:

broker endpoint
queue names
retry policy
visibility timeout
dead-letter configuration
worker concurrency
connection limits

Queue configuration must correspond to the Job & Worker Execution System defined in the preceding stack.

23. EVENT BUS CONFIGURATION

Event infrastructure shall include:

event broker
topic names
consumer groups
retention
delivery mode
retry policy
dead-letter routing

Event configuration shall remain compatible with the OneDraft Integration & Event Bus architecture.

24. WORKER CONFIGURATION

Worker configuration shall control:

worker count
concurrency
queue assignments
job timeout
retry count
heartbeat interval
resource limits
shutdown behavior

Different worker classes may have different configuration profiles.

Example:

GEOMETRY_WORKER
VALIDATION_WORKER
DOCUMENT_WORKER
EXPORT_WORKER
AI_WORKER

25. API CONFIGURATION

API configuration shall control:

- host
- port
- request timeout
- maximum request size
- CORS policy
- rate limits
- authentication
- authorization
- documentation exposure

Production API documentation exposure shall be explicitly controlled.

26. FRONTEND CONFIGURATION

Frontend configuration shall distinguish between:

build-time configuration

and:

runtime configuration

Public frontend configuration may contain:

- API base URL
- application version
- feature availability
- public environment identifier

It must never contain:

- database credentials
- private API keys
- signing keys
- service credentials
- secret provider credentials

27. AI CONFIGURATION

AI configuration shall support provider abstraction.

Example:

- provider
- model
- temperature

token limits
request timeout
retry policy
context limits
tool permissions
evaluation mode

The underlying provider must remain replaceable.

28. ARIA CONFIGURATION

Aria-specific configuration shall include:

command execution mode
model selection
context limits
confidence thresholds
tool permissions
validation requirements
maximum operation scope
provider routing
evaluation configuration

Aria configuration shall not bypass the application command boundary.

29. GEOMETRY ENGINE CONFIGURATION

Geometry configuration may include:

precision
coordinate tolerance
unit system
geometry worker limits
maximum model complexity
calculation timeout
geometry cache policy

The geometry engine must receive configuration appropriate to the active environment.

30. COMPLIANCE ENGINE CONFIGURATION

Compliance configuration shall include:

jurisdiction source configuration

code retrieval settings
manifest generation
cache policy
source validation
effective-date handling

Regulatory data shall remain separate from generic application configuration.

31. DOCUMENT ENGINE CONFIGURATION

Document generation configuration may include:

default paper size
drawing standards
rendering resolution
font configuration
template paths
PDF generation settings
artifact retention
document hashing

Production document-generation configuration shall be version controlled through deployment configuration.

32. FEATURE FLAGS

OneDraft may use feature flags to control controlled functionality.

Examples:

ENABLE_ARIA_COMMANDS
ENABLE_GEOMETRY_ENGINE
ENABLE_CODE_VALIDATION
ENABLE_IFC_EXPORT
ENABLE_DWG_EXPORT
ENABLE_CUSTOM_TEMPLATES
ENABLE_EXTERNAL_INTEGRATIONS

Feature flags must have explicit ownership and lifecycle status.

33. FEATURE FLAG STATES

A feature flag shall support:

ENABLED
DISABLED

Future implementations may support:

PERCENTAGE
ORGANIZATION
PROJECT
USER
ENVIRONMENT

Feature targeting must be deterministic.

34. CONFIGURATION VERSIONS

Significant configuration changes shall be versionable.

A configuration record should identify:

configuration_version
environment
created_at
created_by
change_reason
configuration_hash

This permits reconstruction of runtime conditions.

35. CONFIGURATION HASHING

A normalized configuration representation may be hashed.

Example:

configuration_hash

The hash provides an integrity identifier for the effective configuration state.

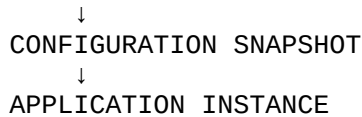
Secrets themselves must not be included in plaintext within the hash record.

36. IMMUTABLE DEPLOYMENT CONFIGURATION

A deployed application instance should operate from a known configuration snapshot.

Conceptually:

DEPLOYMENT



Configuration changes should normally produce a new deployment or controlled runtime configuration update.

37. RUNTIME CONFIGURATION

Runtime configuration may be changed without restarting a service only where explicitly supported.

Examples may include:

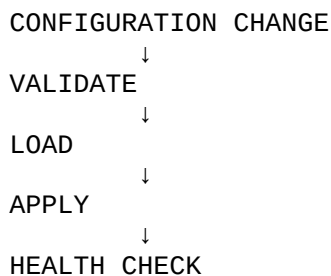
- feature flags
- operational thresholds
- non-secret tuning values
- maintenance mode

Security-sensitive configuration should require controlled deployment or restart.

38. CONFIGURATION RELOAD

Services supporting configuration reload shall expose an internal mechanism.

Conceptually:



An invalid configuration must not replace the currently valid configuration.

39. CONFIGURATION ATOMICITY

Configuration updates shall be atomic.

The system must not enter a state where only part of a configuration update has been applied.

Conceptually:

```
OLD CONFIG
↓
VALIDATE NEW CONFIG
↓
APPLY NEW CONFIG
↓
NEW CONFIG
```

If validation fails:

OLD CONFIG REMAINS ACTIVE

40. CONFIGURATION SCHEMA VALIDATION

Configuration shall be validated for:

```
type
required fields
allowed values
ranges
relationships
environment restrictions
secret references
dependency availability
```

Invalid values shall prevent startup or configuration activation.

41. ENVIRONMENT-SPECIFIC RESTRICTIONS

Certain configuration values shall be restricted by environment.

Example:

```
production
↓
production database only

staging
↓
staging database only

test
↓
test database only
```

A production service must not accidentally connect to development resources.

42. PRODUCTION SAFETY CHECKS

Production startup shall verify:

```
environment == production
database endpoint is authorized
secret provider is available
TLS configuration is valid
debug mode is disabled
development features are disabled
test integrations are disabled
```

The service shall fail closed when mandatory safety conditions are not met.

43. DEBUG MODE

Debug configuration shall be explicitly environment-controlled.

Production shall prohibit:

```
debug tracebacks
development reloaders
development credentials
verbose secret-bearing logs
unsafe development endpoints
```

44. CORS CONFIGURATION

CORS origins shall be explicitly configured.

Development may permit local origins.

Production shall permit only authorized application origins.

Wildcard production origins shall not be used where credentialed requests are supported.

45. TLS CONFIGURATION

Production service communication shall support TLS where required.

Configuration shall include:

certificate reference
private key reference
trusted certificate authorities
minimum protocol
cipher policy

Private certificate keys remain secrets.

46. LOGGING CONFIGURATION

Logging configuration shall define:

log level
structured logging
output destination
retention
redaction
request correlation
service identification

Production logging shall default to structured machine-readable output.

47. OBSERVABILITY CONFIGURATION

The configuration layer shall integrate with the Observability, Telemetry & System Health subsystem.

Configuration shall support:

metrics endpoint
trace exporter
log exporter
sampling
health checks
alerting integration

Telemetry configuration must not expose secrets.

48. SERVICE DISCOVERY

Service endpoints shall be environment-specific.

Example:

API
DATABASE
QUEUE
EVENT BUS

OBJECT STORAGE
GEOMETRY SERVICE
VALIDATION SERVICE
DOCUMENT SERVICE
AI SERVICE

Services shall not hard-code environment-specific hostnames into application logic.

49. EXTERNAL INTEGRATION CONFIGURATION

External integrations shall be configured through isolated provider records.

Examples:

AI providers
payment providers
identity providers
cloud storage
CAD/BIM integrations
email services
webhooks

Each integration shall have its own configuration and secret boundary.

50. CONFIGURATION DEPENDENCIES

Services shall declare their configuration dependencies.

Example:

API

- database
- authentication
- event bus
- configuration

GEOMETRY WORKER

- database
- queue
- object storage
- geometry configuration

A service must not silently depend on undeclared configuration.

51. LOCAL DEVELOPMENT CONFIGURATION

Local development shall support:

```
.env
.env.example
docker-compose.yml
development configuration
```

The repository may contain:

```
.env.example
```

but shall never contain:

```
.env
```

with real credentials.

52. CONFIGURATION TEMPLATE

The repository shall provide a safe configuration template.

Example:

```
.env.example

ONEDRAFT_ENVIRONMENT=development
ONEDRAFT_DATABASE_HOST=localhost
ONEDRAFT_DATABASE_PORT=5432
ONEDRAFT_DATABASE_NAME=onedraft
ONEDRAFT_DATABASE_USER=onedraft
ONEDRAFT_DATABASE_PASSWORD=
ONEDRAFT_STORAGE_ENDPOINT=
ONEDRAFT_QUEUE_URL=
ONEDRAFT_AI_PROVIDER=
ONEDRAFT_AI_API_KEY=
```

Empty secret values indicate that credentials must be supplied externally.

53. SECRET REDACTION

Configuration objects must provide safe representations.

For example:

```
database_password = *****
```

```
api_key = *****  
private_key = *****
```

Sensitive fields must never be rendered through generic object serialization.

54. CONFIGURATION AUDIT

Configuration changes shall produce audit records where configuration is centrally managed.

An audit event may contain:

```
configuration_id  
environment  
actor_id  
change_type  
changed_fields  
timestamp  
configuration_hash
```

Secret values shall never be included.

55. CONFIGURATION OWNERSHIP

Every production configuration domain shall have an identified owner.

Domains include:

```
application  
database  
security  
AI  
workers  
storage  
event_bus  
observability  
billing  
integrations
```

Configuration without ownership should not be promoted to production.

56. CONFIGURATION CHANGE CONTROL

Production configuration changes shall follow:

```
REQUEST  
↓  
REVIEW
```

↓
VALIDATION
↓
DEPLOYMENT
↓
HEALTH CHECK
↓
AUDIT

Emergency changes must remain auditable.

57. CONFIGURATION COMPATIBILITY

Configuration versions must remain compatible with the application version consuming them.

Conceptually:

APPLICATION vX
↓
CONFIGURATION vY

Compatibility shall be explicitly verified during deployment.

58. CONFIGURATION MIGRATIONS

When configuration structures change, migrations may be required.

Examples:

rename configuration field
split configuration field
change default
change provider
remove deprecated setting

Configuration migration procedures shall be versioned alongside the application.

59. DEPRECATED CONFIGURATION

Deprecated configuration values shall have a defined lifecycle:

ACTIVE
DEPRECATED
REMOVED

Deprecation should produce warnings before removal.

60. CONFIGURATION TESTING

Automated tests shall verify:

- required configuration
- environment isolation
- secret references
- default values
- invalid values
- production safety
- configuration compatibility
- feature flags
- service dependencies

61. SECRET TESTING

Automated checks shall verify that:

- secrets are absent from Git
- secrets are absent from Docker images
- secrets are absent from frontend bundles
- secrets are absent from logs
- secret references resolve correctly
- production secrets remain environment-specific

Secret scanning shall be incorporated into CI/CD.

62. CONFIGURATION TEST MATRIX

The initial test matrix shall include:

- Development
- Test
- Staging
- Production

For each environment:

- configuration load
- schema validation
- secret resolution
- database connection
- queue connection
- storage connection
- service startup
- health verification

63. CONTAINER CONFIGURATION

Containers shall receive configuration through controlled mechanisms.

The image itself should remain environment-neutral.

Conceptually:

```
ONE IMAGE
+
ENVIRONMENT CONFIG
+
SECRETS
=
DEPLOYED SERVICE
```

This permits the same application artifact to move through environments.

64. CONTAINER SECRET RULE

Secrets shall not be baked into:

```
Dockerfile
Docker image layers
package files
frontend bundles
source code
```

Secret injection occurs at runtime or deployment.

65. KUBERNETES COMPATIBILITY

The architecture shall remain compatible with Kubernetes-style configuration mechanisms.

Potential resources include:

```
ConfigMap
Secret
Deployment
StatefulSet
Job
CronJob
Service
```

The OneDraft application itself shall remain platform-neutral.

66. CONFIGURATION FOR WORKER AUTOSCALING

Worker scaling parameters may include:

- minimum workers
- maximum workers
- queue depth threshold
- CPU threshold
- memory threshold
- job concurrency
- scale-up delay
- scale-down delay

These values belong to deployment configuration rather than domain state.

67. RESOURCE LIMIT CONFIGURATION

Services may define:

- CPU limits
- memory limits
- request limits
- job execution limits
- file-size limits
- geometry complexity limits
- document generation limits

These limits protect system stability.

68. SECURITY CONFIGURATION

Security configuration shall include:

- token expiration
- issuer
- audience
- authentication providers
- authorization policy
- session limits
- CORS
- CSRF policy where applicable
- encryption requirements
- TLS requirements

Security defaults shall be restrictive.

69. API TOKEN CONFIGURATION

Token validation configuration may include:

- issuer
- audience
- JWKS endpoint
- clock tolerance
- expiration policy
- required claims

The API shall reject tokens that do not satisfy the configured policy.

70. WEBHOOK CONFIGURATION

Webhook configuration shall include:

- endpoint
- signing secret reference
- timeout
- retry policy
- maximum attempts
- delivery event types

Webhook signing secrets remain confidential.

71. BILLING CONFIGURATION

Billing configuration shall support:

- provider
- environment
- product identifiers
- price identifiers
- webhook configuration
- metering configuration
- entitlement configuration

Production billing credentials must remain isolated from development billing credentials.

72. JURISDICTION CONFIGURATION

Jurisdiction-specific configuration shall remain distinct from regulatory source content.

Configuration may define:

jurisdiction identifier
code source providers
default code manifest strategy
municipality integration
effective-date policy

The Compliance & Regulatory Engine remains authoritative for regulatory interpretation and manifests.

73. TEMPLATE CONFIGURATION

Document and drawing templates shall be configurable.

Examples:

ARCH_D
ARCH_E
ARCH_C
custom organization template
jurisdiction template

Templates should be versioned artifacts rather than embedded application constants.

74. CONFIGURATION AND ARTIFACT STORAGE

Configuration references to artifacts shall use immutable artifact identifiers where reproducibility matters.

Example:

template_id
configuration_version
artifact_hash

This integrates with the Artifact & Storage Management Engine.

75. CONFIGURATION AND VERSION CONTROL

Source-controlled configuration shall include only non-secret declarative configuration.

Examples:

defaults
schemas
development templates
feature definitions
service defaults
deployment manifests
configuration documentation

Secrets remain outside source control.

76. CONFIGURATION AND RELEASE ENGINEERING

CI/CD shall validate configuration before deployment.

Conceptually:

```
COMMIT
↓
CONFIGURATION VALIDATION
↓
SECRET SCAN
↓
APPLICATION BUILD
↓
TESTS
↓
DEPLOYMENT VALIDATION
↓
RELEASE
```

A configuration failure shall block release.

77. CONFIGURATION AND OBSERVABILITY

Every service should expose non-sensitive effective configuration metadata sufficient for diagnostics.

Example:

environment
application_version
configuration_version
configuration_hash
service_name

Secret values are excluded.

78. CONFIGURATION AND AUDITABILITY

Operationally significant configuration changes must be traceable to:

who
what
when
why
environment
version
result

This provides configuration provenance without exposing sensitive values.

79. CONFIGURATION RECOVERY

Configuration required to restore a production environment shall be recoverable.

Recovery material shall include:

application configuration
deployment configuration
database connection configuration
storage configuration
queue configuration
event bus configuration
integration configuration
secret references

Actual secret values remain protected within the secret-management system.

80. DISASTER RECOVERY INTERACTION

Disaster recovery shall restore configuration in dependency order:

```
INFRASTRUCTURE
↓
SECRET PROVIDER
↓
DATABASE
↓
STORAGE
↓
QUEUE / EVENT BUS
↓
APPLICATION CONFIGURATION
↓
```

SERVICES



WORKERS

This integrates directly with the Backup, Disaster Recovery & Business Continuity subsystem.

81. CONFIGURATION DRIFT

The system shall detect unexpected differences between declared and deployed configuration.

Conceptually:

DECLARED CONFIG



COMPARE



DEPLOYED CONFIG



DRIFT RESULT

Drift should generate an operational event.

82. CONFIGURATION DRIFT RESPONSE

Configuration drift may result in:

WARNING

BLOCKED DEPLOYMENT

AUTOMATIC RECONCILIATION

OPERATOR REVIEW

The selected behavior shall depend on configuration criticality.

Security configuration should receive the highest protection.

83. CONFIGURATION SNAPSHOT

A deployment should record a configuration snapshot identifier.

Example:

```
deployment_id
application_version
configuration_version
configuration_hash
environment
timestamp
```

This allows operational reconstruction.

84. RUNTIME IDENTITY

Each OneDraft service shall have an identifiable runtime identity.

Example:

```
service_name
environment
instance_id
deployment_id
version
```

This identity integrates with observability, audit and event systems.

85. STARTUP SEQUENCE

The standard service startup sequence shall be:

```
PROCESS START
↓
IDENTIFY ENVIRONMENT
↓
LOAD CONFIGURATION
↓
LOAD SECRET REFERENCES
↓
VALIDATE CONFIGURATION
↓
VALIDATE DEPENDENCIES
↓
INITIALIZE SERVICE
↓
REGISTER TELEMETRY
↓
REGISTER HEALTH CHECKS
↓
READY
```

86. SHUTDOWN SEQUENCE

The service shall support controlled shutdown:

```
STOP ACCEPTING NEW WORK
↓
FINISH OR RELEASE ACTIVE REQUESTS
```

↓
ACKNOWLEDGE OR REQUEUE JOBS
↓
FLUSH TELEMETRY
↓
CLOSE CONNECTIONS
↓
SHUTDOWN

Workers shall not silently discard active jobs.

87. CONFIGURATION FAILURE STATES

Configuration failures shall be classified.

Initial categories:

MISSING_REQUIRED_VALUE
INVALID_VALUE
INVALID_SECRET
INCOMPATIBLE_VERSION
UNAUTHORIZED_RESOURCE
DEPENDENCY_UNAVAILABLE
SECURITY_POLICY_VIOLATION

These errors shall be observable through the system health architecture.

88. ADMINISTRATIVE CONTROL

The Product Configuration & Administrative Control Plane may eventually provide controlled configuration management.

However, administrative interfaces shall operate through the same configuration contracts and authorization boundaries as other OneDraft services.

Administrative interfaces must not bypass configuration validation.

89. CONFIGURATION API

Future internal APIs may expose:

GET /internal/configuration
GET /internal/configuration/version
GET /internal/configuration/health

Sensitive values must never be returned.

These endpoints are operational interfaces, not public client APIs.

90. CONFIGURATION SECURITY BOUNDARY

The final configuration architecture is:

```
SOURCE CONTROL
  ↓
DECLARATIVE CONFIGURATION
  ↓
DEPLOYMENT SYSTEM
  ↓
SECRET PROVIDER
  ↓
RUNTIME CONFIGURATION
  ↓
SERVICE
```

No service may bypass the defined configuration boundary.

91. REPOSITORY STRUCTURE

The implementation shall support configuration resources such as:

```
config/
├── schemas/
├── defaults/
├── development/
├── test/
├── staging/
└── production/
```

```
.env.example
```

```
deploy/
├── development/
├── test/
├── staging/
└── production/
```

Actual production secrets shall not reside within this repository structure.

92. PYTHON CONFIGURATION MODULE

The application shall provide a centralized configuration module.

Conceptually:

```
packages/  
  config/  
    __init__.py  
    settings.py  
    schemas.py  
    loaders.py  
    validators.py  
    secrets.py
```

All application services should consume validated configuration through this boundary.

93. CONFIGURATION OBJECT MODEL

The configuration model should distinguish:

```
ApplicationSettings  
DatabaseSettings  
StorageSettings  
QueueSettings  
EventBusSettings  
WorkerSettings  
AISettings  
SecuritySettings  
ObservabilitySettings  
DocumentSettings  
GeometrySettings  
ComplianceSettings  
BillingSettings  
IntegrationSettings
```

This prevents a single unstructured configuration object from becoming a system dependency bottleneck.

94. CONFIGURATION INJECTION

Services should receive configuration through dependency injection.

Conceptually:

```
service = ProjectModelService(  
    settings=settings,  
    repository=repository,  
    event_bus=event_bus  
)
```

Services should not independently read environment variables throughout the domain layer.

95. DOMAIN LAYER RESTRICTION

The domain model shall remain independent from environment-specific configuration.

The domain layer must not directly depend upon:

- os.environ
- Docker
- Kubernetes
- cloud providers
- secret managers
- deployment systems

Infrastructure adapters translate external configuration into application settings.

96. CONFIGURATION CONTRACT

The configuration system therefore becomes an explicit contract between:

```
SOURCE
↓
BUILD
↓
DEPLOYMENT
↓
RUNTIME
↓
APPLICATION
```

Configuration behavior must be deterministic for a given:

- application version
- configuration version
- environment
- secret set

97. FIRST IMPLEMENTATION ARTIFACTS

The implementation package shall initially contain:

- packages/config/settings.py
- packages/config/schemas.py
- packages/config/loaders.py
- packages/config/validators.py
- packages/config/secrets.py

.env.example

Deployment-specific configuration shall be added under the infrastructure architecture.

98. FIRST AUTOMATED TESTS

The first configuration test suite shall verify:

1. Development configuration loads.
 2. Test configuration loads.
 3. Staging configuration loads.
 4. Production configuration schema validates.
 5. Missing mandatory configuration fails startup.
 6. Invalid configuration fails validation.
 7. Secret values are redacted.
 8. Production cannot use development endpoints.
 9. Configuration hashes are deterministic.
 10. Configuration versions are compatible.
 11. Feature flags resolve correctly.
 12. Service dependencies are validated.
 13. Configuration drift is detectable.
 14. Configuration snapshots are reproducible.
-

99. IMPLEMENTATION SUCCESS CRITERIA

The Configuration, Secrets & Environment Management System is successful when OneDraft can:

Start consistently in every supported environment

Validate all required configuration before becoming ready

Separate development, test, staging and production resources

Load secrets without storing them in source control

Prevent secret exposure through logs and APIs

Version significant configuration

Detect configuration drift

Record configuration provenance

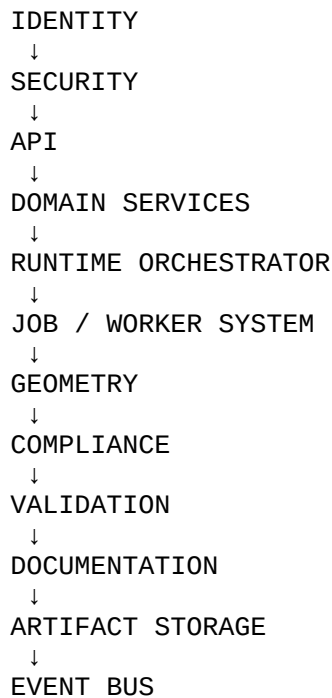
Support controlled configuration changes

Recover the configuration required for disaster recovery

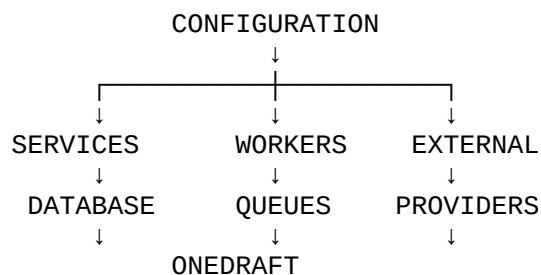
Operate the same application artifact across environments

100. ARCHITECTURAL INTEGRATION

The configuration subsystem now connects the preceding OneDraft architecture:



and provides the environmental control plane beneath them:



101. FINAL ARCHITECTURAL DECISION

OneDraft shall not embed environment-specific behavior directly into application logic.

Instead:

APPLICATION
+
VALIDATED CONFIGURATION
+
AUTHORIZED SECRETS
=
RUNTIME

The application artifact remains environment-neutral.

The environment determines its runtime configuration.

Secrets remain outside the application artifact.

Configuration changes remain controlled, versionable and auditable.

102. SPECIFICATION STATUS

**ONEDRAFT CONFIGURATION, SECRETS & ENVIRONMENT MANAGEMENT SYSTEM
v0.1**

STATUS: ESTABLISHED

The Configuration, Secrets & Environment Management subsystem is now defined as the controlled environmental configuration layer for OneDraft.

It establishes:

Environment separation

Configuration hierarchy

Typed configuration

Secret management

Secret isolation

Runtime configuration

Feature flags

Provider configuration

Service configuration

Configuration versioning

Configuration hashing

Configuration auditability

Configuration drift detection

Deployment compatibility

Configuration recovery

Production safety controls

Configuration testing